



# EkoraChat

Messagerie mobile temps réel avec agent IA conversationnel — Rapport de projet

|                         |                                                                                                       |
|-------------------------|-------------------------------------------------------------------------------------------------------|
| <b>Dépôt GitHub</b>     | <a href="https://github.com/EvansNzati007/EkoraChat">https://github.com/EvansNzati007/EkoraChat</a>   |
| <b>Backend en ligne</b> | <a href="https://ekorachat-production.up.railway.app">https://ekorachat-production.up.railway.app</a> |
| <b>Auteur</b>           | NZATI DOUMBI Evans (Développeur, Génie Logiciel)                                                      |

# 1. Contexte et objectifs

---

EkoraChat est une application de messagerie mobile Android développée dans le cadre d'un projet de fin de formation. L'objectif est de reproduire les mécanismes fondamentaux d'une messagerie instantanée moderne : authentification sécurisée, échange de messages en temps réel, envoi de médias (photos, notes vocales, fichiers), présence en ligne, accusés de lecture, notifications push, et un agent conversationnel basé sur l'intelligence artificielle.

Le projet met en œuvre une architecture client-serveur complète : une application Android native écrite en Kotlin avec Jetpack Compose côté client, et une API backend NestJS déployée sur Railway côté serveur, communiquant à la fois par requêtes REST classiques et par une connexion WebSocket permanente pour le temps réel.

## 2. Architecture générale

L'application est structurée en deux couches communicantes, avec des services tiers dédiés à des besoins spécifiques (notifications, intelligence artificielle) :

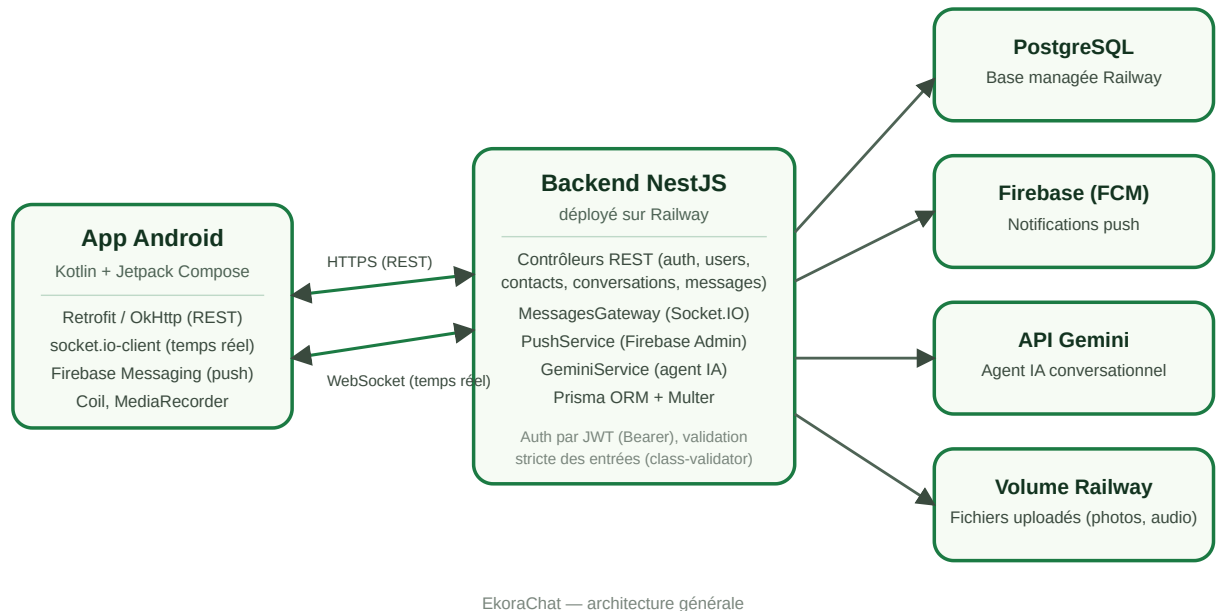


Figure 1 — Architecture générale d'EkoraChat

La communication entre l'application Android et le backend se fait par deux canaux complémentaires : des requêtes REST classiques (authentification, listes de conversations, historique des messages, gestion du profil) et une connexion WebSocket permanente (Socket.IO) pour la diffusion instantanée des nouveaux messages, des accusés de lecture et du statut de présence.

### Stack technique

| Composant                 | Technologies                                                                                                    |
|---------------------------|-----------------------------------------------------------------------------------------------------------------|
| Backend                   | NestJS (TypeScript), Prisma ORM, PostgreSQL, Socket.IO, Multer, Passport-JWT, bcrypt, class-validator           |
| Android                   | Kotlin, Jetpack Compose (Material 3), Retrofit + OkHttp, socket.io-client, Coil, Navigation Compose, Coroutines |
| Notifications             | Firebase Cloud Messaging (Admin SDK côté serveur, Firebase Messaging côté client)                               |
| Intelligence artificielle | API Gemini (agent conversationnel intégré à une conversation dédiée)                                            |
| Déploiement               | Railway (backend + PostgreSQL managé + volume persistant pour les fichiers)                                     |

### 3. Modèle de données

Le schéma relationnel repose sur cinq entités principales : les utilisateurs, leurs contacts, les conversations (privées ou avec l'agent IA), la table de jointure des participants, et les messages échangés.

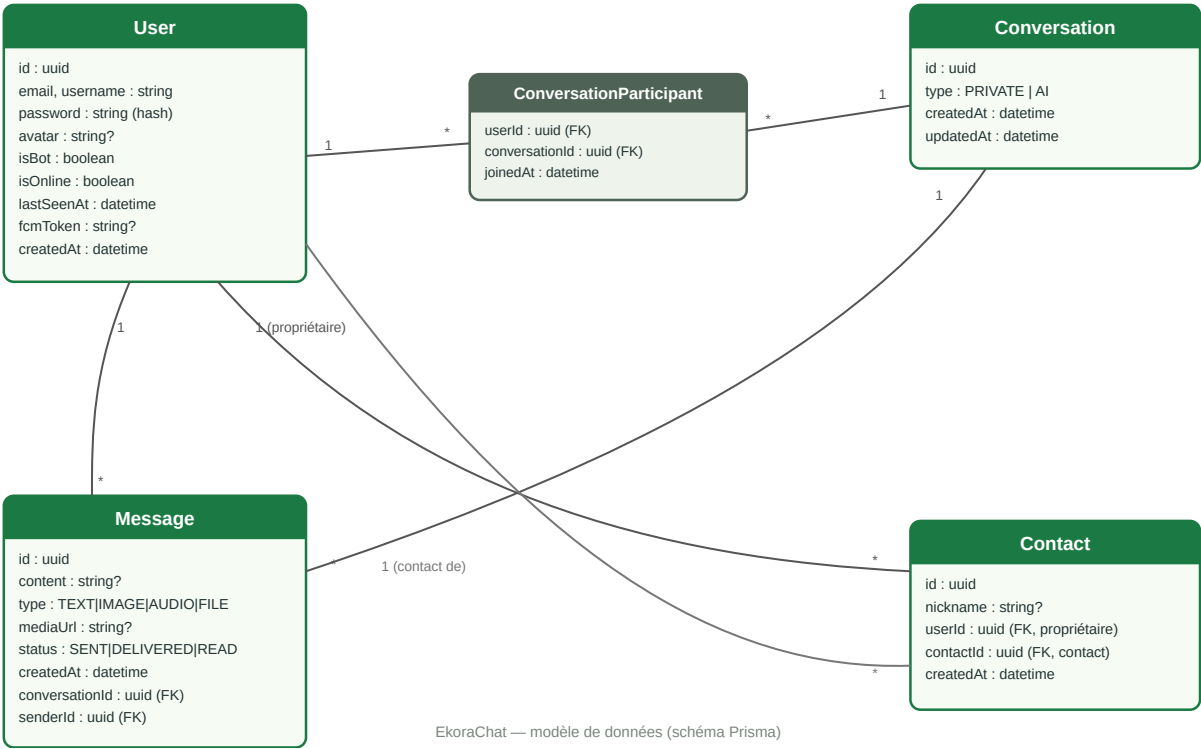


Figure 2 — Modèle de données (schéma Prisma)

## 4. Fonctionnalités principales

---

### Authentification sécurisée

Inscription / connexion par email et mot de passe, JWT, mots de passe hashés (bcrypt).

### Chat en temps réel

Messages diffusés instantanément via WebSocket (Socket.IO), authentifié par JWT au handshake.

### Envoi de médias

Photos, notes vocales (enregistrement intégré) et fichiers, stockés sur un volume persistant.

### Accusés de réception

Statut envoyé / lu mis à jour et diffusé en temps réel à l'ouverture de la conversation par le destinataire.

### Présence en ligne

Indicateur « en ligne » / « vu à HH:mm », basé sur les connexions WebSocket actives.

### Page profil

Modification du nom d'utilisateur, du mot de passe et de la photo de profil.

### Agent IA conversationnel

Conversation dédiée avec un agent basé sur l'API Gemini, intégré comme un utilisateur bot.

### Notifications push

Firebase Cloud Messaging, envoyées uniquement si le destinataire n'a pas déjà la conversation ouverte ; tap sur la notification → ouverture directe de la conversation concernée.

### Formulaires sécurisés

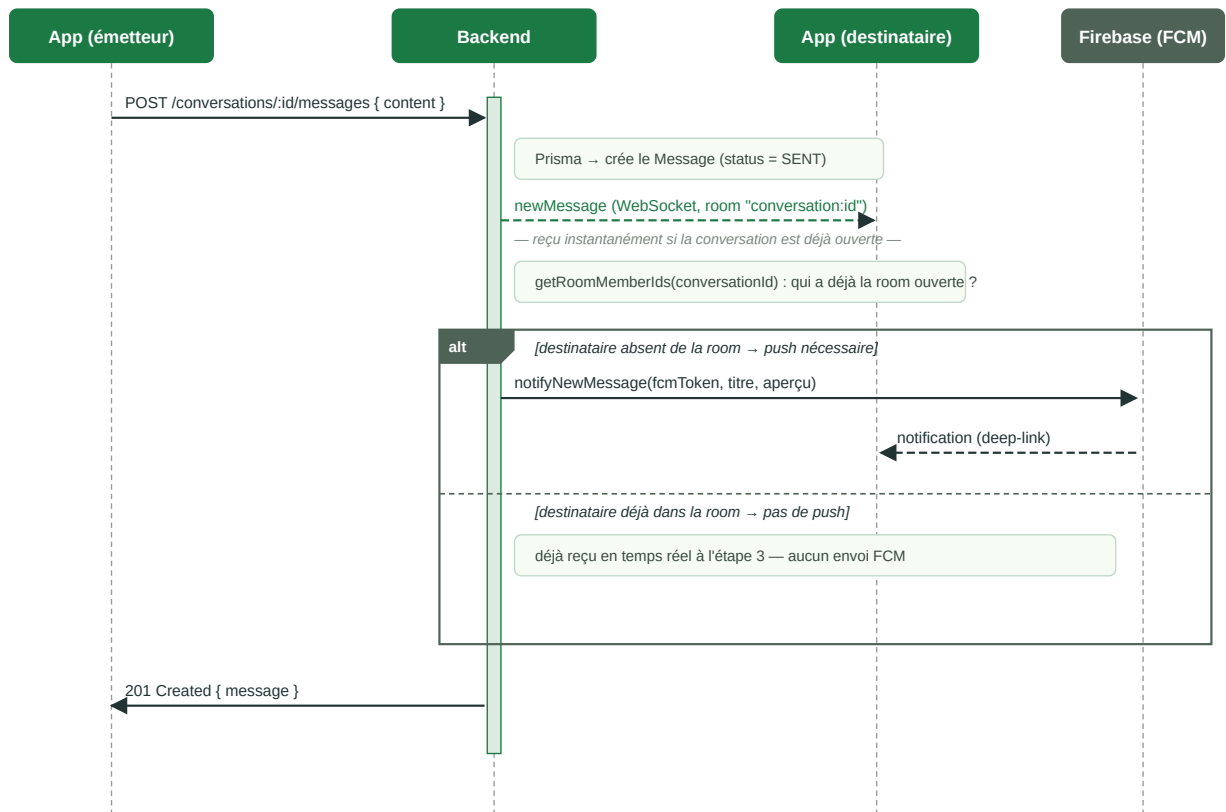
Validation stricte côté serveur (longueurs, formats, caractères autorisés) et côté client (retour immédiat, boutons désactivés tant que les champs ne sont pas valides).

### Thème clair / sombre

L'interface suit automatiquement le thème système du téléphone.

## Flux temps réel vs notification push

Le point le plus intéressant du projet est la coordination entre le canal temps réel et les notifications push, pour éviter les doublons tout en garantissant qu'aucun message ne passe inaperçu :



EkoraChat — envoi d'un message : temps réel ou notification push selon la présence

Figure 3 — Décision temps réel / notification push à l'envoi d'un message

## 5. Sécurité

---

- **Authentication** — JWT signé côté serveur, vérifié sur chaque requête REST (garde d'authentification) et à la connexion WebSocket (handshake).
- **Mots de passe** — hashés avec bcrypt, jamais stockés ni transmis en clair après l'inscription.
- **Validation des entrées** — chaque endpoint valide strictement son corps de requête (email au bon format, longueurs minimales et maximales, rejet de tout champ non attendu) grâce à class-validator / class-transformer.
- **Restriction de format** — le nom d'utilisateur est limité à un jeu de caractères sûr (lettres, chiffres, `._-`), empêchant l'injection de contenu arbitraire (ex. balises de script).
- **Limites anti-abus** — taille maximale des fichiers uploadés, longueur maximale du contenu des messages, whitelist des types MIME acceptés.
- **Secrets** — clés Firebase et variables sensibles jamais commitées, injectées via variables d'environnement Railway.

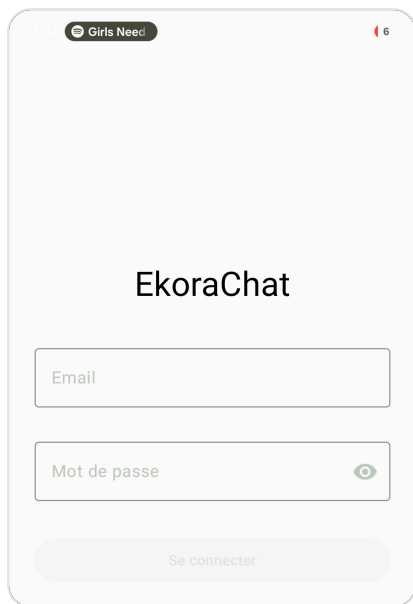
## 6. Déploiement

---

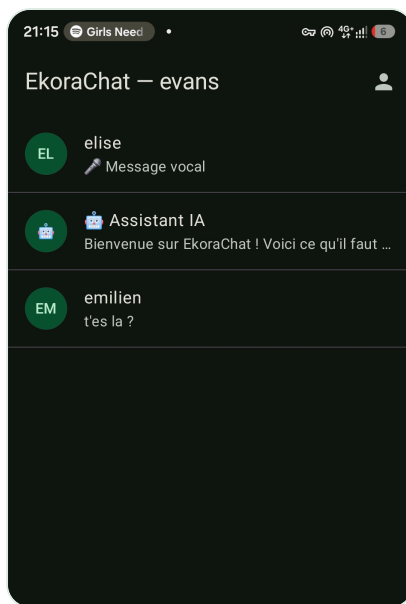
Le backend est déployé en continu sur Railway à partir du dépôt GitHub : chaque merge sur la branche suivie déclenche un nouveau déploiement, qui applique automatiquement les migrations de base de données en attente ( `prisma migrate deploy` ) avant de démarrer le serveur. Les fichiers médias envoyés par les utilisateurs sont conservés sur un volume Railway persistant, indépendant du cycle de vie des conteneurs.

L'application Android est installée directement via `./gradlew installDebug` sur un appareil connecté en USB (débogage USB activé), sans nécessiter de publication sur un store pour les besoins de la démonstration.

## 7. Captures d'écran



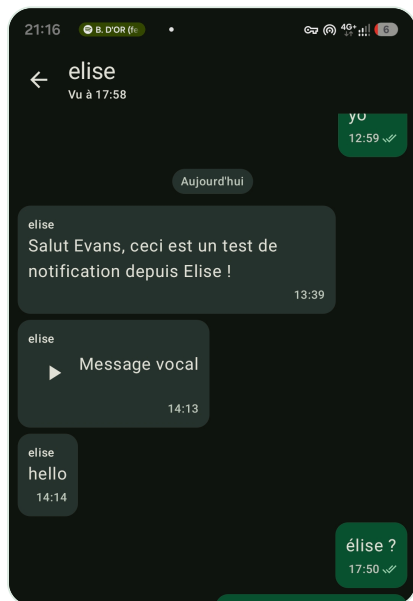
Connexion



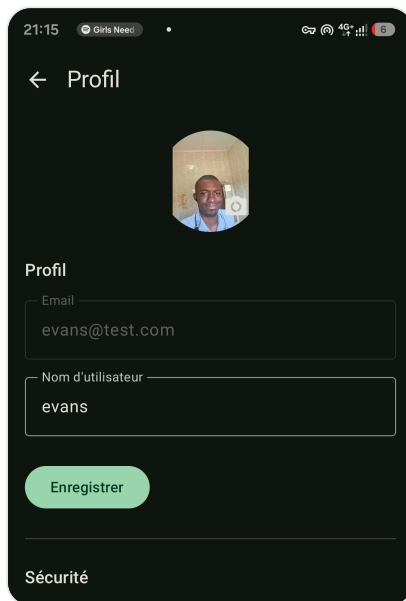
Liste des conversations



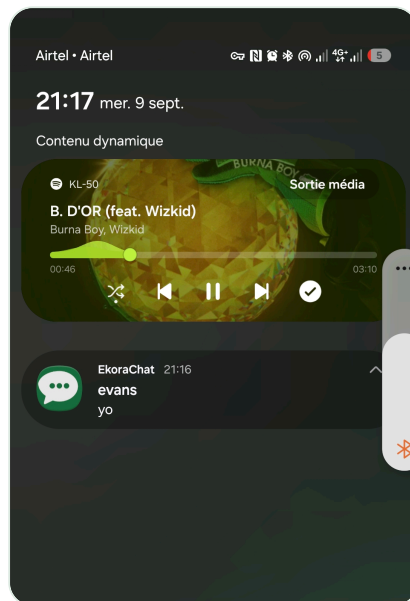
Conversation (chat)



Messages vocaux



Profil



Notification push

## 8. Limites connues et perspectives

---

- Le statut `DELIVERED` est prévu dans le modèle de données mais non exploité (seuls `SENT` et `READ` sont utilisés, pour rester simple).
- Pas d'indicateur « est en train d'écrire » (l'infrastructure WebSocket le permettrait facilement).
- Pas de vérification de disponibilité du nom d'utilisateur en direct pendant la saisie (le rejet arrive à la soumission du formulaire).
- Le chiffrement de bout en bout n'est pas implémenté.

### Liens

- Dépôt GitHub : [github.com/EvansNzati007/EkoraChat](https://github.com/EvansNzati007/EkoraChat)
- Backend en production : [ekorachat-production.up.railway.app](https://ekorachat-production.up.railway.app)